

# AIce - HowTo

## (AKS-Interface)

Dr. Maximilian Hanusch - Landkreis Prignitz

2026-10-04

### [I] Übersicht:

AIce (AKS-Interface) dient als R-Schnittstelle für die AKS und ist als R5 class (Reference class) umgesetzt, d.h., Instanzen der aice.rc-Klasse fungieren als Schnittstelle zur Kaufpreissammlung (AKS-Niedersachsen), die das einfache Laden von Kauffalldaten sowie deren Selektion - bspw. via exportierter Selektionsansätzen - einfach macht.

### Hintergrund:

- Mögliche Änderung an der AKS-Niedersachsen (Export der Kauffalldaten, Änderung der Elemente, etc.) können bereits in den Interner des Interface (AIce) abgefangen werden, sodass der darauf basierende Quellcode (Auswertungen der Kauffalldaten in den Geschäftsstellen, etc.) von diesen Änderungen unberührt bleibt.
- AIce ist in Base R geschrieben - benutzt also keine weiteren Pakete, und ist daher auch resistent gegen etwaige Änderungen solcher Zusatzpakete, bzw. deren Überalterung. Nur mögliche Änderungen am R-Kern selbst müssen bei der Wartung des Interfaces berücksichtigt werden. AIce ist damit weitestgehend wartungsfrei.
- Das Format der Selektionsparameter ist komplett analog zur AKS. Zudem ist das direkte Einlesen exportierter AKS-Selektionsansätze möglich, sodass die in der AKS benutzten Selektionsansätze ohne Weiteres in R übernommen werden können.

Dies soll den Einstieg in R (den Umstieg von AKS auf R) möglichst optimal erleichtern.

## (1) Einbinden und Initialisieren:

Konzeptionell ist es so angedacht, dass der “aice”-Ordner einfach in den entsprechenden aktuellen R-Hauptprojektordner kopiert wird - im Folgenden mit *<PathToProject>* bezeichnet - in dem die eigenen R-Skripte liegen, die AIce benutzen sollen/wollen.

Im entsprechenden Skript wird die AIce-Funktionalität einfach via

```
source("<PathToProject>/aice/aice.R")
```

eingebunden (irgendwo am Anfang des Skriptes hinzufügen und ausführen).

**Hinweis:** Beim Öffnen eines R-Projektes legt R-Studio üblicherweise als Arbeitsverzeichnis automatisch *<PathToProject>* fest, womit Folgendes ebenfalls funktioniert (sofern sich der aice-Ordner direkt in diesem Arbeitsverzeichnis befindet):

```
source("aice/aice.R")
```

Die Etablierung der eigentlichen Schnittstelle findet wie folgt statt:

```
# Instanziierung der aice.rc - R5-Klasse
#
mein_aice <- aice.rc$new()
#
## oder gleichbedeutend:
# mein_aice <- aice.rc$new(TRUE)
```

Es werden dann automatisch die AKS-**exportierten** (siehe Abschn. (2) unten) Kauffall-daten in das AIce-Objekte “mein\_aice” geladen und die Daten intern in die Grundstücksarten (“bb”, “ei”, “ub”, “lf”, “gf”, “sf”) aufgeteilt. Entsprechend automatisch durchgeführte Datenbereinigungsroutinen (Geisterspalten entfernen, Typenuntreue korrigieren) sind in [II].(4) beschrieben.

## Zusatz - vgl. Abschn. [II].(2).(C)

Via

```
mein_aice <- aice.rc$new(FALSE)
```

lässt sich ein leeres AIce-Objekt erstellen, mit dem sich dann u.a. Direktselektionen vormals abgespeicherter und in R eingelesener Datensätze durchführen lassen (siehe Abschn. [II].(3).(C).[c],[d]).

## (2) Daten, Export, Pfade:

### **Allgemein gilt für die AKS:**

- Momentan exportiert die AKS Kauffalldaten als CSV-Dateien mit “;” als Trenner - AIce erwartet dies auch so. Selektionssansätze müssen dann ebenfalls mit “;” als Trenner exportiert werden - vgl. Abschn. [I].(2).(B).
- Prinzipiell exportiert die AKS wohl in der Codierung “ANSI - Windows1252” und per default ist AIce darauf eingestellt - die Codierung ist in der Metadatei

– <ProjectPath>/aice/meta.R

festgelegt, via der Konstanten

– AKS\_FILE\_ENCODING

und kann dort auch geändert werden (man müsste dann die abgespeicherten Kauffalldaten und Selektionsansätze vor der Benutzung entsprechend umwandeln.)

### **Allgemein gilt für AIce:**

Der Hauptablageort der Daten ~ Kauffalldaten / Selektionsansätze / abgespeicherte Selektionen  
~ wird durch die Variable

- m\_PathRootDat

festgelegt, die sich in der folgenden Metadatendatei befindet:

- <ProjectPath>/aice/meta.R

Dort können auch weitere Einstellungen vorgenommen werden:

- Der Ablageort der Kauffalldaten wird spezifiziert durch die Variable
  - m\_PathKaufDat
- Der Ablageort der Selektionsansätze wird spezifiziert durch die Variable
  - m\_PathSelektAns
- Der Ablageort von durch AIce abgespeicherte Selektionen wird spezifiziert durch die Variable
  - m\_PathSelektionen

## (A) Kauffalldaten exportieren und einbinden:

- **Der Export der Kauffalldaten aus der AKS gestaltet sich wie folgt:**

AKS → Allgemeine Auswertung → Gesamtausgabe für uf/bb/ei als CSV-Datei

- **Das Häkchen setzen bei:**

### “Die Werte der Textfelder in Anführungszeichen setzen”

*Hintergrund: Bei manchen Felder mit Textinhalt (wie AFTN) wird andernfalls der Inhalt/Text NICHT in der Form “<Text>” exportiert. Enthält nun <Text> ein “;”-Zeichen, so werden die dadurch separierten Abschnitte beim Laden als Inhalte unterschiedlicher Felder interpretiert (denn “;” ist der standard CSV-Trenner beim Export). Wäre bspw. <Text> eine Zeichenkette der Form “Nr. 1; Nr. 2” im Feld AFTN, so würde*

- diesem Feld beim Import nur der String “Nr. 1” als Inhalt zugeordnet werden,
- dem Feld AMAU (beim Export die Spalte rechts daneben) der String ” Nr. 2” als Inhalt zugeordnet werden.

Die Inhalte der Felder verschieben sich alle nach rechts.

- Den Zeitraum festlegen und “Start” drücken. Die Daten werden von der AKS unter “Q/ausgabe/abgabedateien/...” abgelegt.
- **Das Einbinden der Kauffalldaten in AIce geschieht per default wie folgt:**

Die exportierten Kauffalldaten müssen abgelegt werden unter:

- “<ProjectPath>/aice/data/Kauffalldaten”

Hier bitte nur einen Export-Datensatz ablegen (das sind 3 CSV-Dateien ~ ‘uf’, ‘bb’, ‘ei’), da AIce sonst quasi nach dem “Zufallsprinzip” für jede GRUA einen Datensatz auswählt.

- **Datenpfad ändern:**

Der default-Ablageort der Kauffalldaten kann vermöge den folgenden Variablen in der Datei “.../aice/meta.R” geändert werden (vgl. oben):

- m\_PathRootDat
- m\_PathKaufDat

## (B) Selektionsansätze exportieren und einbinden:

- Der Export der Selektionsansätze aus der AKS geschieht per default wie folgt:

Auftrag in der “**Allgemeinen Auswertung**” in der AKS:

Auftrag → Ausgabe Selektionsansatz → Ausgabe Netto-Selektionsansatz

Hier “Wertetabelle” sowie “Semikolon (;)” als Trenner wählen und abspeichern.

- **!Hinweis!:** Man kann durch Klicken des “Umkopieren”-Buttons den Ablageort gleich auswählen.
- Das Einbinden der Selektionsansätze in AIce geschieht wie folgt:

Der default-Ablageort der (exportierten) Selektionsansätze ist:

– “<ProjectPath>/aice/data/Selektionsansätze”

- Datenpfad ändern:

Der default-Ablageort der Selektionsansätze kann vermöge den folgenden Variablen in der Datei “.../aice/meta.R” geändert werden (vgl. oben):

– m\_PathRootDat  
– m\_PathSelektAns

## (3) Selektieren:

### Version 1 ~ Open-Dialog:

Die einfachste Art und Weise mit AIce zu selektieren ist:

```
df_Selection <- mein_aice$select("")
```

### Ablauf:

- Es öffnet sich ein Open-Dialog-Fenster mit dem man den gewünschten Selektionsansatz einfach auswählen kann (standardmäßig befindet man sich bereits im default-Ablagepfad für die Selektionsansätze).
- Im data.frame df\_Selection wird die selektierte Liste abgespeichert.
- Es werden in der Konsole entsprechende Informationen ausgegeben.

### Version 2 ~ Pfad:

Man kann auch den kompletten Pfad (inklusive Dateiname) angeben bzw. den Pfad relativ zum aktuellen Arbeitsverzeichnis:

```
df_Selektion <- mein_aice$select("<Pfad_zum_abgespeicherten_Selektionsansatz>")
df_Selektion <- mein_aice$select("./<RelativPfad_zum_abgesp_Selektionsansatz>")
```

### Version 3 ~ ASel = AKS-Selektionsansatz:

Man kann “`aice.rc$select()`” ein sogenanntes ASel-Objekt übergeben. Auch beim Laden von Selektionsansätzen wird dieses intern erzeugt und für die Selektion genutzt; dann aber wieder vernichtet, sodass man es nicht zu Gesicht bekommt. Es handelt sich bei ASel ebenfalls um eine R5-Klasse. Details folgen später - hier nur kurz ein paar Anmerkungen zur Erzeugung eines solchen Objekts:

```
## leeres ASel erzeugen:
# NULL-ASel ~ komplett leer:
mein_asel <- asel.rc$new(NULL)
# leer, aber mit festgelegter Grundstücksart:
mein_asel <- asel.rc$new(<grua>)
# mit <grua> in {"bb","ei","ub","lf","gf","sf"}
#
## ASel erzeugen und mit abgespeichertem Selektionsansatz befüllen:
# via Open-Dialog:
mein_asel <- asel.rc$new()
# via vollst. Pfad zum Selektionsansatz:
mein_asel <- asel.rc$new("<Pfad_zum_abgespeicherten_Selektionsansatz>")
```

Selektiert wird dann via:

```
df_Selektion <- mein_aice$select(mein_asel)
```

Bezüglich `<grua>` ist anzumerken, dass man einem ASel-Objekt ~ außer bei der Verwendung im Kontext von Filtern (Abschn. [II].(1)) und Direktselektionen (Abschn. [II].(3).(C).[c]) immer eine Grundstücksart zuweisen muss - und zwar damit AIce ~ weiß, in welchem Teildatensatz es selektieren soll.

**!Beachte!:** Beim Laden von Selektionsansätzen wird die korrekte GRUA automatisch aus dem Dateinamen extrahiert - daher sollte man diese nicht beliebig umbenennen!

- Bspw. erhält man bei der Verwendung eines NULL-ASels eine entsprechende Warnmeldung in der Konsole:

```
mein_asel <- asel.rc$new(NULL)
```

Leeres asel.rc-Objekt erzeugt; keine GRUA festgelegt ~ nutze: asel\$setGrua().

```
df_Selection <- mein_aice$select(mein_asel)
```

AKS-Selektionsobjekt asel.rc-Objekt uebergeben.

NULL

Warnmeldungen:

1: In aselOb\$getGrua() :

Noch keine GRUA festgelegt: 'bb', 'ei', 'ub', 'lf', 'gf', 'sf'

2: In aice\$select(asel) :

Fehlende GRUA ~ keine Selektion durchgefuehrt ~ 'NULL' zurueckgegeben.

- Bei der Verwendung eines leeren (nicht-NULL) ASels erhält man (wie erwartet) die komplette Kauffallliste in der entsprechenden Gundstücksart zurück:

```
mein_asel <- asel.rc$new("bb")
```

```
df_Selection <- mein_aice$select(mein_asel)
```

AKS-Selektionsobjekt asel.rc-Objekt uebergeben.

Inversionsflag gesetzt: 'FALSE'

??? Kauffaelle selektiert.

- Ändert man die GRUA via asel.rc\$setGrua(<grua>), so werden die Selektionsparameter bzgl. der entsprechenden Sonderregel (Sonderkatalog) intern neu erstellt.

#### (4)> Picken (typecasting):

AIce lädt und hält die Daten im String-Format. Die Rückgaben von “select(...)” ist per default ge-typecasted:

```
df_Selektion <- mein_aice$select(..., TPCST = TRUE)
```

Die Rückgaben von “getSelda(...) und getData(...) (vgl. Abschn. [II].(2))” sind aber per default im Stringformat. Um die Tabellenspalten in ihren entsprechenden Datentypen umzuwandeln, kann man das (typecast) “TPCST”-Flag explizit setzen:

```
df_Selektion <- mein_aice$getSelda(..., TPCST = TRUE)
df_Selektion <- mein_aice$getData( ..., TPCST = TRUE)
```

oder die entsprechenden “pick”-Methoden verwenden - **Details in Abschn. [II].(4).[D-F]**. Bspw. reduziert

```
tpcst_Selekt <- mein_aice$pick(df_Selektion, c("KFKZ", "DATU"))
```

“df\_Selektion” auf die Spalten “KFKZ” und “DATU” und typecasted diese in den entsprechenden Datentypen. Allgemein ist:

```
aice.rc$pick(dFrm = NULL, vIKs = c(), grua = "", TPCST = TRUE)
```

- Lässt man den zweiten Slot leer (oder übergibt c() / NULL), so werden alle Spalten konvertiert und zurückgegeben.
- *Momentan obsolet:*  
Stehen gewisse Spaltenbezeichner in einer der Sonderkataloge, so kann man als weiteren dritten Parameter noch die GRUA übergeben.
- Setzt man TPCST = FALSE , so verbleiben die entsprechenden Einträge im String-Format.

## [II] Mehr zum AIce:

### (1) Filter - Vorselektion:

Die Dauer einer Selektionsoperation steigt merklich mit der Anzahl der Kauffalldaten. AIce bietet daher die Möglichkeit der “Vorselektion”, d.h., das Setzen eines Filters via einem ASel-Objekte. Selektiert wird dann im Folgenden automatisch nur noch in den vorselektierten (gefilterten) Daten. Die verfügbaren Operationen sind:

- `aice.rc$setFltr(<fltr_asel>)` ~ setze Filter
- `aice.rc$rmvFltr(<grua>)` ~ remove/entferne Filter

#### (A) `aice.rc$setFltr(aselOb = NULL)`:

```
# Sei <fltr_asel> ein asel.rc-Objekt:
## setzt <fltr_asel> als Filter fest:
mein_aice$setFltr(<fltr_asel>)
# Ist in <fltr_asel>
#   eine GRUA festgelegt, so wird der Filter nur für diese GRUA gesetzt.
#   keine GRUA festgelegt, so wird der Filter für alle GRUAs gesetzt.
#
## Filter via Open-Dialog-Auswahl setzten (Selektionsansatz laden):
mein_aice$setFltr()
```

#### (B) `aice.rc$rmvFltr(grua = NULL)`:

```
# Filter in der Grundstücksart <grua> in {"bb","ei","ub","lf","gf","sf"}
# entfernen:
mein_aice$rmvFltr(<grua>)
#
# Filter in allen Grundstücksarten entfernen:
mein_aice$rmvFltr()
```

#### (C) Beispiel:

```
ZEITRAUM <- "01.01.2025:31.12.2025"
# leeres ASel mit GRUA = "bb" erzeugen:
fltr_asel <- asel.rc$new("bb")
# dem ASel einen neuen Selektionsparameter hinzufügen ~ DATU:
# Details zu "addSepa" später in Abschn. [III]
fltr_asel$addSepa("DATU", ZEITRAUM)
#
# Filter setzen:
mein_aice$setFltr(fltr_asel)
```

## (2) Kauffalldaten-Handling:

### (A) `aice.rc$getData(grua = "", FILTER = TRUE, TPCST = FALSE)`

- Gibt die importierten Kauffalldaten als `data.frame` zurück. Per default ist “FILTER = TRUE”:

| <b>FILTER</b> | <b>Rückgabe</b>                                     |
|---------------|-----------------------------------------------------|
| TRUE          | Gefilterte Kauffalldaten in der Kategorie “grua”.   |
| FALSE         | Ungefilterte Kauffalldaten in der Kategorie “grua”. |

- TPCST = TRUE (TPCST = TYPECAST):  
Die Spalten werden in den entsprechenden Datentyp umgewandelt zurückgegeben.
- Ist keine GRUA angegeben, so wird eine Warnmeldung ausgegeben.

```
# BEISPIEL:  
#  
# gleichbedeutend mit $getData("bb", TRUE), $getData("bb", FILTER = TRUE)  
data <- mein_aice$getData("bb")  
#  
# gleichbedeutend mit ...$getData('bb', FILTER = FALSE):  
data <- mein_aice$getData("bb", FALSE)
```

### (B) `aice.rc$setData(data = NULL, grua = "", FILTER = TRUE, CHTPCST = TRUE)`

Mit dieser Methode kann man per “Gewalt” die Datengrundlage neu festlegen. Per default ist gesetzt:

- CHTPCST = TRUE, sodass die übergebenen Daten (genau wie beim Laden mit `loadData(...)`) im Stringformat (`character`) abgespeichert werden.
- FILTER = TRUE, sodass die “Vorselektions-Daten” in der Kategorie `<grua>` ad hoc durch `<data>` ersetzt werden:

```
# sei <data> ein data.frame mit Kauffalldaten:  
# gleichbed. mit: mein_aice$setData("bb", <data>, FILTER = TRUE)  
mein_aice$setData(<data>, "bb")
```

Setzt "FILTER = FALSE", so wird die geladen Hauptdatenbank (natürlich nur im AIce-Objekt) überschrieben. Auch mit `aiice.rc$rmvFltr(...)` lassen sich dann die Ursprungsdaten nicht mehr zurückholen - man muss dann die Daten per Hand neu laden:

### (C) `aiice.rc$loadData(pathKaufDat = "")`

Lädt die Kauffalldaten in das AIce-Objekt:

```
mein_aice$loadData(<myPath>)
```

Will man bspw. Kauffalldaten aus einer anderen als der Standardquelle laden, so kann man dies wie folgt bewerkstelligen:

```
# gegeben ein <Pfad_zu_anderen_Daten>
#
# neues AIce-Objekt erzeugen:
mein_anderes_aice <- aiice.rc$new(loadData = FALSE)
      # gleichbed.: aiice.rc$new(FALSE)
#
mein_anderes_aice$loadData("<Pfad_zu_anderen_Daten>")
```

**Hinweis:** Bei der Erzeugung eines aice-Objektes in der Form:

```
aiice.rc$new() # gleichb. zu: aiice.rc$new(loadData = TRUE)
```

wird `aiice.rc$loadData(...)` intern einfach in der Form

```
.self$loadData(m_PathKaufDat)
```

aufgerufen - sie entfernt automatisch alle Filter.

### (3) Selektionsdaten-Handling:

#### (A) Laden und Speichern von Selektionsdaten:

AIce stellt die folgenden Methoden zum Speichern (`write.table(...)`) und Laden (`read.table(...)`) von (selektierten) Daten bereit:

##### [a] `aice.rc$wtfData(...)` - "WriteToFileDaten"

```
aice.rc$wtfData(data = NULL, fPath = NULL, grua = "",  
                TPCST = TRUE, ENCOD = STD_FILE_ENCODING)
```

- `data` ~ `data.frame` mit den selektierten Daten
- `fPath` ~ kompletter Pfad zum Abspeichern  
Bei "NULL"-Übergabe ~ Open-Dialog im Pfad, spezifiziert durch "m\_PathSelektionen"
- Falls gültige "grua", so wird diese zum Dateinamen hinzugefügt, sofern noch nicht enthalten.
- `TPCST` ~ wenn `TRUE`, dann werden die Spalten vor dem Abspeichern in ihren entsprechenden Datentyp umgewandelt (AICE lädt und hält alle Daten im String-Format).
- `ENCOD` ~ Dateikodierung ~ Default ist "UTF-8"

##### [b] `aice.rc$lffData(...)` - "LoadFromFileDaten"

```
aice.rc$lffData(fPath = NULL, ENCOD = STD_FILE_ENCODING, TPCST = FALSE, grua = "")
```

- Gibt die geladenen Daten als `data.frame` zurück.
- `fPath` ~ kompletter Pfad zum Laden - Open-Dialog bei "NULL"-Übergabe
- `ENCOD`: Dateikodierung ~ Default ist "UTF-8"
- `TPCST`: Spalten in entsprechenden Datentyp umwandeln?
- `grua`: Falls `TPCST = TRUE` sagt `grua`, mit welchem Sonderkatalog ge-typecastet werden soll.

### [c] aice.rc\$wtfSelda(...) - “WriteToFileSelektionsdaten”

```
aice.rc$wtfSelda(Index = 0, fPath = NULL, TPCST = TRUE)
```

Diese Methode speichert direkt aus der internen Selektionshistorie ab (siehe Punkt (B) unten) - die GRUA wird im Dateinamen automatisch vermerkt.

- Index ~ Index der durchgeführten Selektion
- fPath ~ kompletter Pfad zum Abspeichern - Open-Dialog bei “NULL”-Übergabe.
- TPCST ~ wenn TRUE, dann werden die Spalten vor dem Abspeichern in ihren entsprechenden Datentyp umgewandelt (AICE lädt und hält alle Daten im String-Format).

### [d] aice.rc\$wtfFltrList(...) - “WriteToFileFilterliste”

```
aice.rc$wtfFltrList(fname = "FltrDat", MERGE = TRUE, TPCST = FALSE)
```

Speichert die vorselektierten Kauffalldaten im “Selektionen-Ordner” ab. Ist “MERGE = TRUE”, so werden die unbebauten Flächen in einen Datensatz gepackt. Die GRUA wird im Dateinamen vermerkt.

Beispielsweise ist dann die Ausgabe von

```
mein_aice$wtfFltrList()  
# erzeugt: FltrDat_bb.csv, FltrDat_ei.csv, FltrDat_uf.csv
```

genau in dem Format, welches “aice.rc\$loadData(...)” als Input erwartet.

### (B) Selektionsdaten Intern - getSelda(...) und getSelma(...)

```
getSelda(Index = 0, META = FALSE, TPCST = FALSE)    und    getSelma(Index = 0)
```

- Wenn TPCST = TRUE, dann werden die Spalten in den entsprechenden Datentyp umgewandelt zurückgegeben.

Bei jeder Selektion via “aice.rc\$select(...)” werden die selektierten Daten intern von AIce in einer fortlaufenden Liste abgespeichert:

```

### Gibt die Anzahl der abgespeicherten Selektionen zurück:
Anz <- mein_aice$getAnzSel()
#
### Selektionsdaten als data.frame zurückgeben:
## Gibt die letzte Selektion zurück:
df <- mein_aice$getSelda()
# Gleichbedeutend zu:
df <- mein_aice$getSelda(0)
df <- mein_aice$getSelda(Index = 0)
## Gibt die 3-te Selektion zurück (Warnmeldung, falls außerhalb des Bereichs.)
df <- mein_aice$getSelda(3)
# Gleichbedeutend zu:
df <- mein_aice$getSelda(Index = 3)

```

Nun hat `getSelda(...)` noch den Parameter “META”, der standardmäßig auf ‘FALSE’ gesetzt ist. Wählt man “META = TRUE”, so erhält man als Rückgabewert eine Liste der Form:

```
List <- list(GRUA ~ character, MASK ~ logical-vector, NAs ~ data.frame)
```

Den zweiten Eintrag “MASK” erhält man auch mit:

```

# get-Selektionsmaske:
blnMask <- mein_aice$getSelma(<IND>)

```

### Bedeutung der Listenelemente:

Es bezeichne “ZGK” den zugrundeliegenden Kauffalldatensatz als `data.frame`, aus dem selektiert (bei der Selektion Nr. `<IND>`) - festgelegt durch die ursprünglich geladenen Daten, der GRUA und den zugehörigen (möglicherweise trivialen) Filter:

- `List$GRUA` ~ enthält die Grundstücksart, in der selektiert wurde
- `List$MASK` ~ Vektor in den Einträgen {TRUE, FALSE, NA} (Details in Abschn. (C))

Die Einträge mit TRUE sind genau die positiv selektierten Kauffälle, d.h., der Rückgabewert von `getSelda(<IND>)` ist gerade gegeben durch:

```

ZGK[which(List$MASK),]
# bzw. ZGK[which(blnMask),]
# bzw. ZGK[which(mein_aice$getSelma(<IND>)),]

```

- `List$NAs` ~ `data.frame` - Kauffälle, die bei der Selektion schon alleine wegen fehlender Werte rausgefallen sind.

Entsteht aus “ZGK” wie folgt:

- Von  $ZGK$  wird die Spalte “KFKZ” behalten; und zudem jede Spalte zu einer Ausprägung, nach der  
**ohne die Verwendung der Metakriterien ! / = (ist belegt / nicht belegt)** selektiert wurde.  
Alle anderen Spalten werden entfernt  $\rightarrow$  dies definiert ein reduziertes data.frame  $ZGK_{RED}$
- Es werden aus  $ZGK_{RED}$  nur die Zeilen behalten, die mindestens einen Eintrag “NA” haben.

### (C) Selektionsvarianten:

**Im Folgenden ist zu beachten**, dass **R** im Kontext von logischen Operationen, 'NA's als (zwar) UNBEKANNTE (aber dennoch) LOGISCHE Werte behandelt:

| logisches Symbol | Bedeutung |
|------------------|-----------|
| !                | NICHT     |
| &                | UND       |
|                  | ODER      |

**Hiermit gilt:** (man lege besonderes Augenmerk auf die Zeilen “**3**” und “**6**”)

Table 1: **TABELLE 1: BOOLSCHER-LOGIK**

| Ausdruck            | logischer Wert |
|---------------------|----------------|
| 1 ! NA              | NA             |
| 2 NA & NA           | NA             |
| <b>3</b> NA & FALSE | FALSE          |
| 4 NA & TRUE         | NA             |
| 5 NA   NA           | NA             |
| <b>6</b> NA   TRUE  | TRUE           |
| 7 NA   FALSE        | NA             |

### [a] Selektion via logischer Maske:

```
aice.rc$selectMask(grua = "", mask = NULL, TPCST = FALSE)
```

!!! Die Selektion wird **NICHT** intern in AIce abgespeichert !!!

Mit “**selectMask(...)**” kann man via einem logischen Vektor “mask” (Einträge in (TRUE, FALSE, NA)) in einer Grundstücksart “grua” aus dem zugrundeliegenden Kauffalldatensatz “ZGK” selektieren. Man ist selbst dafür verantwortlich, dass “mask” so viele Einträge hat wie Zeilen (Kauffälle) in “ZGK” vorhanden sind.

**Bspw.** kann man sich wie in (B) beschrieben via

```
retList <- mein_aice$getSelda(<IND>, META = TRUE)
grua <- retList$GRUA
blnMask <- retList$MASK
```

oder einfacher, falls GRUA bekannt via:

```
grua <- <bekannte_GRUA>
blnMask <- mein_aice$getSelma(<IND>)
```

einen entsprechenden Vektor verschaffen, und dann per Hand selektieren via:

```
df <- mein_aice$selectMask(grua, blnMask)
```

Man erhält in diesem Fall natürlich das gleiche Ergebnis wie mit:

```
mein_aice$getSelda(<IND>)
```

Gedacht ist diese Option prinzipiell für den Fall, in dem man mehrere Selektionsmasken logisch miteinander verknüpfen will:

**Bspw.**, angenommen, es wurde via “mein\_aice” im Vorfeld ANZ-mal in der Grundstücksart “bb” selektiert:

```
blnMasks <- list()
ANZ <- mein_aice$getAnzSel()
grua <- "bb"
for (i in 1:ANZ) {
  blnMasks[i] <- mein_aice$getSelma(i)
}
```

```
# Alle Kauffälle, die jeden Selektionsansatz erfüllen:
blnMask <- blnMasks[1]
for (j in 2:n) {
  blnMask <- blnMask & blnMasks[j]
}
df <- mein_aice$selectMask(grua, blnMask)

# Alle Kauffälle, die mindestens einen Selektionsansatz erfüllen:
blnMask <- blnMasks[1]
for (j in 2:n) {
  blnMask <- blnMask | blnMasks[j]
}
df <- mein_aice$selectMask(grua, blnMask)

# Alle Kauffälle, die NICHT die 1. Selektion erfüllen (und keine NAs sind):
df <- mein_aice$selectMask(grua, !blnMasks[1])

# etc.....
```

## [b] INVERSIONS-FLAG

```
aice.rc$select(aselObjPth = NULL, INVERT = FALSE, TPCST = TRUE)
```

- Wenn TPCST = TRUE (default), dann werden die Spalten in den entsprechenden Datentyp umgewandelt zurückgegeben.

Die “**select()**” - Methode bietet die Möglichkeit, als zweiten Parameter das INVERSIONS-FLAG zu setzen.

Konkret ist die “normale” Selektion (wie gehabt) gegeben durch:

```
mein_aice$select(<asel>/<Pfad_zum_ASel>)
# gleichbedeutend mit:
# mein_aice$selekt(<asel>/<Pfad_zum_ASel>, INVERT = FALSE)
```

## Benutzt man

```
df <- mein_aice$select(<aselObj>/<Pfad_zum_ASel>, INVERT = TRUE)
```

so wird intern die logische Selektionsmaske invertiert/negiert **!und auch in dieser Form intern abgespeichert!**; also Vorheriges liefert das Gleiche wie:

```

mein_aice$select(<aselObj>/<Pfad_zum_ASel>, INVERT = FALSE)
#
retList <- mein_aice$getSelda(0, META = TRUE)
  grua <- retList$GRUA
blnMask <- retList$MASK
  df <- mein_aice$selectMask(grua, !blnMask)

```

**Wie bereits erwähnt:** Setzt man bei der Selektion: “INVERT = TRUE”, so wird intern auch die invertierte Maske abgespeichert, d.h., es gilt

```
blnInvMask == !blnMask
```

wenn man setzt:

```

mein_aice$select(<aselObj>/<Pfad_zum_ASel>, INVERT = TRUE)
#
blnInvMask <- mein_aice$getSelma(0)
#
# oder
retList <- mein_aice$getSelda(0, TRUE)
blnInvMask <- retList$MASK

```

### [c] Direktselektion:

```

aice.rc$selectDirect(data = NULL, aselObjPth = NULL, INVERT = FALSE,
                     META = FALSE, TPCST = FALSE)

```

!!! Die Selektion wird **NICHT** intern in AIce abgespeichert !!!

Bedeutungen:

| Parameter          | Bedeutung                                                |
|--------------------|----------------------------------------------------------|
| aselObjPth, INVERT | wie bei aice.rc\$select()                                |
| META               | wie bei aice.rc\$getSelda()                              |
| data               | hier wird ein Kauffalldatensatz als data.frame übergeben |

Die Rückgabe von “**aice.rc\$selectDirect(...)**” ist ein data.frame mit den selektierten Kaufalldaten. Es wird hier **NICHT** auf die GRUA im übergebenen/geladenen ASel-Objekt geprüft. Es werden die übergebenen Kaufalldaten “data” stur selektiert - man ist selbst dafür verantwortlich, dass alles zusammenpasst.

#### [d] MASK-Direktselektion:

```
aice.rc$selectMaskDirect(data = NULL, mask = NULL, TPCST = FALSE)
```

!!! Die Selektion wird **NICHT** intern in AIce abgespeichert !!!

Bedeutungen:

| Parameter  | Bedeutungen                      |
|------------|----------------------------------|
| mask, data | wie bei aice.rc\$selectMask(...) |

Der Rückgabewert ist banalerweise:

```
return(data[which(mask),])
```

#### (4) Elementekatalog - der “meta”-Ordner:

Im Ordner “/aice/meta” liegt der Elementekatalog:

- **befüllt:** “HauptKTG.csv”
- **momentan noch leer:** “SonderKTG\_bb/ei/ub/lf/gf/sf.csv”

Dieser wird von AIce intern genutzt, um die ElementeIDs (welche in den abgespeicherten Selektionsansätzen verwendet werden) mit den Kurzbezeichnungen (welche in den exportierten Kauffalldaten verwendet werden) zueinander in Beziehung zu setzen. Zudem ist für jedes Element ein AIce-interner Datentypbezeichner (SPT) hinterlegt (AIce weiß dann, auf welche Art zu selektieren ist). Schließlich gibt es noch das Singleton-Flag, welches diejenigen Periodenhauptelemente via “1” markiert, die selbst auch belegt sein koennen (FLAC, BOWE). Diese werden im Kontext von Selektionen als ganz normale eigenstaendige Felder behandelt (d.h., es wird NICHT via ODER über deren Perioden-Unterelemente getestet).

Man erhält die Kataloge via:

```
mein_aice$getHptKatalog()      # Hauptkatalog
mein_aice$getSndKatalog(<grua>) # Sonderkataloge
# mit <grua> in {"bb","ei","ub","lf","gf","sf"}
```

Die Sonderkataloge existieren prinzipiell deswegen, weil die AKS intern anscheinend - zumindest für gewisse Elemente - in den verschiedenen Grundstücksarten unterschiedliche **IDs** vergibt - siehe bspw.

- AKS → Infos → Übersicht Freie Felder / Übersicht Weitere Merkmale

Genauer wird bei diesen Perioden eine weitere Ziffer angefügt (“1” für “ub” ...); **aber** in den exportierten Selektionsansätzen tauchen diese momentan nur in der verkürzten (GRUA-unspezifischen Form) auf, sodass eine Spezifizierung momentan noch nicht nötig ist. Fall sich das ändert, so würde man die entsprechenden Elemente in “Hauptkatalog.csv” löschen und dann in jede Sonderkatalog-Datei mit der entsprechenden zusätzlichen Ziffer einfügen.

#### **Beachte ~ EDAT:**

Im Hauptkatalog wurde die Kurzbezeichnung der Periode 111 “EDAT” in “\_\_**EDAT**” abgeändert, da das Unterelement 1111 ebenfalls “EDAT” heißt.

#### **Beachte ~ Geisterfelder:**

Die AKS hält wohl historische Elementfelder wie “WGWB” (bzw. “WGWB1”,...,“WGWB5”), die nicht mehr befüllt werden koennen, aber beim Export in der CSV auftauchen ~ und dann einen “Rechtssshift” im Datensatz verursachen. ~ Diese Verschiebungen werden in `func_aice_load_Data(...)` per default mit dem Flag “NoGhosts = TRUE” beim LAaden automatisch korrigiert. Diese werden in den “Geister”-CSV-Dateien im Ordner “aice/meta/Data-Cleanups” hinterlegt.

#### **Beachte ~ Typenuntreue:**

Die AKS exportiert manche Felder im falschen Format; zumindest jedenfalls “ZGEZ” bei “ub” und “sf”. Dies ist ein Integer-Feld, welches als Dezimalzahl mit zusaetzlichen zwei 0-Nachkommastellen exportiert wird. Dies wird in `func_aice_load_Data(...)` per default mit dem Flag “TypeRepair = TRUE” beim Laden automatisch korrigiert. Die Felder zusammen mit der entsprechenden Korrekturoperation werden in den “TypeRepair”-CSV-Dateien im Ordner “aice/meta/DataCleanups” hinterlegt.

#### **Beachte ~ Leerspalten:**

Die AKS exportiert (zumindest momentan) mit einem zusätzlichen “;”-CSV-Trenner am Ende jeder Zeile. “read.table” interpretiert das als unbenannte Spalte, und gibt ihr den Namen “X”. Eine entsprechende Bereinigung kann bspw. in der Definition von “`func_aice_load_data()`” via “NoUnnamed = TRUE” aktiviert werden.

**[A]** `aice.rc$lotoK(vlKs = c(), grua = "") ~ ID onto KB`

Transformiert in beide Richtungen:

- Elemente-ID  $\longleftrightarrow$  Elemente-KB

Wird “grua”

- *weggelassen*, so wird der Hauptkatalog benutzt.
- *angegeben*, so wird der Hauptkatalog zusammen mit dem Sonderkatalog benutzt.

Momentan liefern also beide Varianten noch das gleiche Ergebnis.

```
## Ohne "grua": mein_aice$IotoK(c("KFKZ", "DAT")) # [1] 101 201 # mein_aice$IotoK(c(101, 201))
```

**!Beachte:** Der Einfachheit halber ist vIKs als Vector implementiert (nicht als Liste), sodass man entweder IDs oder KBs angeben kann - keine Mischform.

```
[B] aice.rc$getElids(grua = "")
```

## Die Methode

```
mein_aice$getElids(<grua>)
# mit <grua> in {"bb","ei","ub","lf","gf","sf"}
```

liefert ein einzeliges `data.frame`:

- Die Spaltennamen (= Elementekurzbezeichnungen) sind (in gleicher Reihenfolge) die des (zu <grua>) gehörigen (geladenen) Datensatzes.
- Zeile 1 enthält die jeweils zugehörige ELeemente-ID ~ extrahiert aus dem Hauptkatalog sowie dem zugehörigen Sonderkatalog.

```
[C] aice.rc$getCollInds(vlKs = c(), grua = "")
```

Liefert die Spaltennummern im entsprechenden (grua) Kauffalldatensatz, die zu den übergebenen IDs/KBs gehören:

```
aiice.rc$getColInds(c("KFKZ","DAT"),"bb")
```

**[D] `aice.rc$pick(dfrm = NULL, vIKs = c(), grua = "", TPCST = TRUE)`**

Extrahiert die zu vIKs (IDs oder KBs) gehoerigen Spalten aus "dfrm" (data.frame).

- Wenn `TPCST = TRUE` (typecast), dann werden die Spalten entsprechend in ihren Datentyp konvertiert.
- Wenn `vIKs = c() / NULL`, dann wird das gesamte dFrm konvertiert zurückgegeben.

**[E] `aice.rc$pickFiltr(vIKs = c(), grua = "", TPCST = TRUE) ~ pick Filter`**

Extrahiert aus den vorselektierten (gefilterten) Daten in der Kategorie "grua" die durch den ID/KB-Vektor "vIKs" spezifizierten Spalten ~ reduziert also den Datensatz auf die gewünschten Elemente (TPCST wie oben).

**[F] `aice.rc$pickSelda(vIKs = c(), Index = 0, TPCST = TRUE) ~ pick Selektionsdaten (data.frame)`**

Extrahiert aus der entsprechenden internen Selektion (Index), die durch den ID/KB-Vektor "vIKs" spezifizierten Spalten ~ reduziert also den Datensatz auf die gewünschten Elemente (TPCST wie oben).

**[G] `aice.rc$typeCast(data = NULL, grua = "", <...>) ~ Helpermethode`**

Wandelt die Spalteneinträge vom data.frame "data" in ihren entsprechenden Datentyp um (AICE lädt und hält alle Daten im String-Format). Werden IDs statt KBs uebergeben, so werden diese - falls `grua = ""` uebergeben wird - alleine aus dem "Hauptkatalog" umgerechnet; ansonsten zudem aus dem entsprechenden Sonderkatalog. Die volle Parameterliste ist:

```
aice.rc$typeCast(dFrm = NULL, grua = "", NoEntryErrMsg = FALSE,  
                 SptErrMsg = FALSE, TPCST = TRUE)
```

wobei die letzten 3 Parameter in den folgenden "Wartungsmethoden" benutzt werden:

**[H] `aice.rc$checkKtgs(GRUAS = "", NoEntryErrMsg = TRUE, SptErrMsg = TRUE)`**

Prüft auf fehlende Elementeeinträge in den Katalogen, sowie fehlende SPT-Einträge bei den Elementen, die in den geladenen Daten als Spaltennamen vorkommen.

### [I] aice.rc\$checkData(GRUAS = "", FLTR = TRUE)

Prüft auf verbliebene Typecastfehler, die dann bspw. auf mögliche Fehlbelegungen von Feldern hinweisen (die entsprechenden Elemente, Werte und Kauffallnummern werden in der Konsole ausgegeben).

### [III] Mehr zum ASel:

Wir erinnern an Abschn. [I].(3). Dort hatten wir unter “**Version 3**” bereits die Existenz der sogenannten ASel-Objekte diskutiert ~ die zugehörige R5-Klasse heißt “asel.rc”:

```
### leeres ASel erzeugen:  
# NULL-ASel ~ komplett leer:  
mein_asel <- asel.rc$new(NULL)  
# leeres ASel, aber mit festgelegter Grundstücksart:  
mein_asel <- asel.rc$new(<grua>  
                        # mit <grua> in {"bb","ei","ub","lf","gf","sf"})  
  
### ASel erzeugen und mit abgespeichertem Selektionsansatz füllen:  
# via Open-Dialog:  
mein_asel <- asel.rc$new()  
# via vollst. Pfad zum Selektionsansatz:  
mein_asel <- asel.rc$new("<Pfad_zum_abgespeicherten_Selektionsansatz>")
```

Wir hatten gesehen, dass man “aice.rc\$select(...)” anstatt eines Pfades zu einem abgespeicherten AKS-Selektionsansatz (\*.csv), genauso gut auch ein ASel-Objekt übergeben kann.

Bspw. liefern die folgenden beiden Varianten das gleiche Ergebnis:

```
# A:  
df <- mein_aice$select("<Pfad_zum_abgespeicherten_Selektionsansatz>")
```

```
# B:  
mein_asel <- asel.rc$new("<Pfad_zum_abgespeicherten_Selektionsansatz>")  
df <- mein_aice$select(mein_asel)
```

Der eigentliche Sinn der Benutzung von ASel-Objekten ist es natürlich, via R Selektionsansätze per Hand zu kreieren, bzw. geladene Selektionsansätze (via AKS abgespeicherte CSV-Dateien) noch per Hand nachzujustieren. Die entsprechenden Methoden werden wir nun diskutieren.

## (1) Allerlei:

### (A) nochmal das Inversions-Flag:

Der zusätzliche Parameter “INVERT” in

```
aice.rc$select(aselObjPth = NULL, INVERT = FALSE)
```

ist ausschließlich für die Situation gedacht, in der “aselObjPth” ein Pfad zu einem abgespeicherten Selektionsansatz ist und kein separates ASel-Objekt. In letzterem Fall wird das “INVERT”-Flag in select(...) ignoriert, und man muss

```
# Setzt das Inversionsflag:  
mein_asel$setInv(TRUE/FALSE)  
#  
# Gibt das Inversionsflag zurück:  
mein_asel$getInv()
```

benutzen, sodass bspw. gilt:

```
# Es liefert das gleiche Ergebnis:  
# A:  
df <- mein_aice$select("<Pfad>", INVERT = TRUE)  
#  
# B:  
mein_asel <- asel.rc$new("<Pfad>")  
mein_asel$setInv(TRUE)  
df <- mein_aice$select(mein_asel)
```

### (B) Printen ~ asel.rc\$prtSela(): (a friend)

Gibt den kompletten Selektionsansatz strukturiert in der Konsole aus:

```
mein_asel <- asel.rc$new()  
# ...  
mein_asel$prtSela()
```

Man kann damit prüfen, ob ASel die eingegebenen Selektionsparameter korrekt verarbeitet/interpretiert hat.

## (C) Laden und Speichern per Hand

### [a] `asel.rc$loadSela(pathKaufDat = "")`

Selektionsansatz per Hand laden:

```
mein_asel <- asel.rc$new(NULL)
mein_asel$loadSela("" / "<Path>")
```

### [b] `asel.rc$wtfSela(fPath = "")`: "WriteToFileSelektionsansatz"

Man kann ein ASel-Objekt als CSV-Datei abspeichern (Analog zur AKS):

```
mein_asel.wtfSela(<Path>)
# oder mit Open-Dialog:
mein_asel.wtfSela()
```

**Beachte:** Im Gegensatz zur AKS werden beim Abspeichern die Symbole '!' / '=' anstelle der Textbausteine 'ist belegt' / 'nicht belegt' benutzt.

## (2) Selektionsparameter festlegen/löschen/ändern:

### (A) `asel.rc$getSepaCnt()`

Gibt die Anzahl der bereits festgelegten Selektionsparameter im ASel-Objekt zurück.

### (B) `asel.rc$addSepa(IdKb = NA, InputStr = "", plnd = 0)`

Fügt dem Selektionsansatz (ASel-Objekt) einen weiteren Selektionsparameter (Selektionskriterium) hinzu. Sei

```
ANZ <- asel.rc$getSepaCnt()
```

die Anzahl der bereits festgelegten Selektionsparameter:

| Parameter | Beschreibung                                                                          | Wert                           | Aktion                                                                     |
|-----------|---------------------------------------------------------------------------------------|--------------------------------|----------------------------------------------------------------------------|
| idKb      | der Identifier (als INTEGER) oder die Kurzbezeichnung (als "STRING") des AKS-Elements |                                |                                                                            |
| InputStr  | der entsprechende AKS-Selektions-Formatstring                                         |                                |                                                                            |
| pInd      | interner Index des Kriteriums im ASel-Objekt                                          | 0 = standard                   | Es wird ein neues Kriterium hinten dran gehängt.                           |
|           |                                                                                       | $1 \leq \dots \leq \text{ANZ}$ | Es wird das Kriterium Nr. "pInd" durch das neue Kriterium <b>ersetzt</b> . |

```
# BEISPIEL:
#
# In MINDESTENS einer der WEIA-Perioden ist der Teilstring "SW23" enthalten:
mein_asel$addSepa("WEIA", "*SW23*")

# In KEINER der WEIA-Perioden ist der Teilstring "SW23" enthalten:
mein_asel$addSepa("WEIA", "<>(*SW23*)")

# Mindestens eine der WEIA-Perioden ist belegt:
mein_asel$addSepa("WEIA", "!")

# Keine der WEIA-Perioden ist belegt:
mein_asel$addSepa("WEIA", "=")

# In WEIA7 steht genau der Vollstring "SW23":
mein_asel$addSepa("WEIA7", "SW23")

# WEIA7 ist nicht belegt, oder enthält den Teilstring "SW23":
mein_asel$addSepa("WEIA7", "= # *SW23*")
```

```
# DATU ist nicht belegt oder enthält ein Datum zwischen '01.07.2025:31.12.2025':
mein_asel$addSepa("DATU", " = # 01.07.2025:31.12.2025")

# DATU (ID 201) ist außerhalb von '01.07.2025:31.12.2025':
mein_asel$addSepa(201, "<>(01.07.2025:31.12.2025)")
mein_asel$addSepa(201, "<>01.07.2025:31.12.2025")
# mehr zum '<>()' - Operator in Punkt [d]
#      - beachte hierfür, dass '<>' nur für numerische Felder gilt
```

Insbesondere kann also AIce immer in allen Teilperioden ('WEIA1',..., 'WEIA8') separat selektieren.

#### [a] Metakriterien ~ '!' / '='

- ! (ist belegt) und = (nicht belegt) werden im Folgenden als **Metakriterien** bezeichnet.

#### [b] Singletons:

- Singletons (BOWE, FLAC) gelten im folgenden Kontext **NICHT** als Perioden.

#### [c] Perioden und reguläre Kriterien:

Oben hatten wir:

```
# In MINDESTENS einer der WEIA-Perioden ist der Teilstring "SW23" enthalten:
mein_asel$addSepa("WEIA", "*SW23*")
#
# In KEINER der WEIA-Perioden ist der Teilstring "SW23" enthalten:
mein_asel$addSepa("WEIA", "<>(*SW23*)")
```

Anhand der **zweiten** Selektion im Code sehen wir:

1. Der Negationsoperator negiert das Gesamt-Test-Resultat (also die ge-ODER-ten Wahrheitswerte der Tests auf den Unterelementen "WEIA1",..., "WEIA8").
2. Er testet also **NICHT** das Kriterium "In MINDESTENS einer WEIA-Periode ist NICHT der Teilstring "SW23" enthalten". Um dieses (immanent sinnlose) Kriterium zu selektieren, kann man die folgenden Tricks verwenden:

1. Man benutzt die Logikregel

$$(!A_1) \mid \dots \mid (!A_n) \quad \hat{=} \quad !(A_1 \ \& \ \dots \ \& \ A_n)$$

indem man wie folgt selektiert:

```
for (i in 1:8) {  
  mein_asel$addSepa(paste0("WEIA",i), "*SW23*")  
}  
mein_asel$setInv(TRUE)  
df <- mein_aice$select(mein_asel)
```

2. Man selektiert jedes WEIA<i> (<i>=1,...8) einzeln via

```
mein_asel$addSepa("WEIA<i>", "<>(*SW23*)")
```

verschafft sich die Selektionsmasken `blnMasks[< i >]` (wie in Abschn. [II].(3).(C).[a]) und verknüpft diese via ODER (also '|'), und nutzt dann

```
mein_aice$selectMask(...)
```

3. Hat man es mit einem numerischen Feld zu tun, so kann man auch einfach den normalen UNGLEICH-Operator '<> ' verwenden:

```
mein_asel$addSepa("LAGE", "<>11")
```

**!!! Im Kontext von Perioden liefert also '<> 11' durchaus etwas Anderes als '<> (11)' !!!**

#### [d] Perioden und Metakriterien:

Für Perioden wie **WEIA** gelten die im obigen Codeabschnitt dargestellten 'Sonderregelungen':

1. Selektiert man nach dem Hauptelement "WEIA", so gilt dieses als

- **belegt** ("!=")  
wenn **mindestens eines** der Unterelemente "WEIA1",..., "WEIA8" belegt ist.
- **nicht belegt** ("=")  
wenn **keines** der Unterkriterien "WEIA1", ..., WEIA8 belegt ist.

```
# MINDESTENS eine der WEIA-Perioden ist belegt:  
mein_asel$addSepa("WEIA", "!")  
#  
# KEINE der WEIA-Perioden ist belegt:  
mein_asel$addSepa("WEIA", "=")
```

2. Selektiert man direkt nach einem Unterelement - bspw. WEIA7, so gelten die **üblichen Regeln**:

```
# Das Element WEIA7 ist belegt
mein_asel$addSepa("WEIA7","!")

# Das Element WEIA7 ist nicht belegt
mein_asel$addSepa("WEIA7","=")
```

### [e] Syntax von “InputStr”:

Die Syntax von **InputStr** ist prinzipiell dieselbe wie in der AKS. Zusätzlich gibt es aber den

- **NEGATIONSOPERATOR:**  $\langle \rangle$  (...)

der auch zusammen mit “=” benutzt werden darf (ran-ge-ORDER-t).

Zudem darf an “=” auch ein regulärer **AUSDRUCK** wie

- $A_1 \# \dots \# A_n$  bzw.  $\langle \rangle (A_1 \# \dots \# A_n)$

ran-ge-ORDER-t werden. Es gelten die folgenden Regeln:

1. “**IST BELEGT**” steht für sich alleine - der Rest wird von ASel entfernt:

$$! \# \cancel{\text{AUSDRUCK}} \rightarrow !$$

2. An “**NICHT BELEGT**” darf ran-ge-ORDER-t werden:

$$= \# \text{AUSDRUCK}$$

3. Zusätzlich zur üblichen Negation “ $\langle \rangle$ ” (numerische Elemente / Datum), gibt nun den Erweiterten Negationsoperator:

$$\langle \rangle (A_1 \# \dots \# A_n)$$

- Er zählt ebenfalls als regulärer Ausdruck im Kontext von Punkt 1 und Punkt 2 (wie oben auch bereits angedeutet).
- Außer “=” wird alles bis auf den “ersten (linksstehenden) Negator”  $\langle \rangle$  (...) entfernt:

$$\begin{aligned} &= \# \cancel{\text{AUSDRUCK}_1} \# \langle \rangle (\dots_1) \# \cancel{\text{AUSDRUCK}_2} \# \cancel{\langle \rangle (\dots_2)} \dots \\ &\quad \rightarrow \\ &= \# \langle \rangle (\dots_1) \end{aligned}$$

### (C) `asel.rc$updtSepa(pInd = 0, InputStr = "")`

Es muss gelten:

$$1 \leq \text{pInd} \leq \text{asel.rc\$getSepaCnt}()$$

Der Unterschied zu

$$\text{addSepa}(<\text{IdKb}>, \text{InputStr}, \text{pInd})$$

ist, dass `updtSepa(pInd, InputStr)` die Elemente-ID (genauer die Kurzbezeichnung) `<IdKb>` vor dem Ersetzen des Kriteriums durch "InputStr" aus dem vorhandenen Eintrag von selbst extrahiert und übernimmt.

### (D) `asel.rc$rmvSepa(vInd = 0)`

Entfernt die durch den Index-Vektor 'vInd' spezifizierten Kriterien. Die größeren Elemente rücken Indexmäßig nach (keine Indexlücken). Erlaubt sind:

- eine Aufzählung von Indices:

$$\text{vInd} = c(I_1, \dots, I_k) \text{ mit } 1 \leq I_1, \dots, I_k \leq \text{getSepaCnt}()$$

- etwas von der Form:

$$\text{vInd} = c(I_0, \text{Inf}) \text{ mit } 1 \leq I_0 \leq \text{getSepaCnt}()$$

was alle Kriterien mit Index  $\geq I_0$  löscht.

## (3) SELEKTIONS-LOGIK:

Wie in der AKS gilt, dass der Wahrheitswert einer Selektion durch '&'-Verknüpfung der Wahrheitswerte der verschiedenen Selektionsparameter (Kriterien) gebildet wird. Für jeden Selektionsparameter (der natürlich zu einem bestimmten AKS-Element gehört) - wie bspw.

```
mein_asel$addSepa("DATU", "01.07.2025:31.12.2025")
# oder
mein_asel$addSepa("DATU", "!")
#oder
mein_asel$addSepa("DATU", "= # 02.03.2026")
# etc.
```

gelten separat jeweils die folgenden Wahrheitsregeln:

## (A) LOGIK - Innerhalb eines Selektionsparameters

Es ist zielführend sich klarzumachen, wie der Wahrheitswert innerhalb eines Selektionsparameters (Kriteriums) zustande kommt - insbesondere der "NA"-Fall:

1. Die **Metakriterien** "!" / "=" liefern stets "TRUE" oder "FALSE" - niemals "NA".
  - "!" steht immer alleine - AIce entfernt alles Andere vor der Selektion
  - "=" darf via "#" mit einem "regulären" Kriterium/Ausdruck kombiniert werden.
2. Gegeben ein **reguläres** Kriterium (NICHT-META-Kriterium)

$$K_{reg} \quad \widehat{=} \quad K_1 \# K_2 \# \dots \# K_n$$

oder

$$K_{reg} \quad \widehat{=} \quad <> (K_1 \# K_2 \# \dots \# K_n)$$

auf einem AKS-Element  $E$ , so ist gleichbedeutend:

- das AKS-Element  $E$  ist nicht belegt (ist "NA"),
- jedes Teilkriterium  $K_i$  für  $1 \leq i \leq n$  liefert "NA",
- das Gesamtkriterium  $K_{reg}$  liefert "NA".

3. Gegeben ein **Mischkriterium** (mit  $K_{reg}$  wie in Punkt 2.)

$$K_{misch} \quad \widehat{=} \quad = \# K_{reg}$$

auf einem AKS-Element  $E$ , so gilt

- für den Fall, dass das Element **belegt** ist:

| Status Element: | "=" ist<br>zwangsweise: | K-regulär darf<br>sein: | K-misch hat dann<br>den Wert: |
|-----------------|-------------------------|-------------------------|-------------------------------|
| belegt          | FALSE                   | TRUE                    | TRUE                          |
| belegt          | FALSE                   | FALSE                   | FALSE                         |

- für den Fall, dass das Element **nicht belegt** ist:

|                 | “=” ist<br>zwangsweise: | K-regulär ist<br>zwangsweise: | K-misch ist<br>zwangsweise: |
|-----------------|-------------------------|-------------------------------|-----------------------------|
| Status Element: |                         |                               |                             |
| nicht belegt    | TRUE                    | NA                            | TRUE                        |

**Der Anteil  $K_{reg}$  spielt in diesem Fall also einfach gar keine Rolle.**

Somit liefert im Kontext von Mischkriterien die Selektion innerhalb des Selektionsparameters immer entweder “TRUE” oder “FALSE” (und zwar in sinniger Weise) - und der Wert “NA” tritt niemals auf.

## (B) LOGIK zwischen den Selektionsparametern

Besteht ein Selektionsansatz aus mehreren Selektionsparametern - was im Regelfall so ist - wie bspw.

```
mein_asel$addSepa("WEIA","!")
mein_asel$addSepa("WEIA7","*SW23*")
mein_asel$addSepa("DATU"," = # 01.07.2025:31.12.2025")
...
```

so werden die einzelnen Wahrheitswerte via logischem **UND** verknüpft.

- Diese Werte erhält man als logischen Vektor (jeder Eintrag entspricht einem Kauffall) via (vgl. Abschn. [II].(3).(B)):

```
blnMask <- mein_aice$getSelma(<IND>)
# bzw. umständlicher:
# retList <- mein_aice$getSelda(Index = <IND>, MASK = TRUE)
# blnMask <- retList$MASK
```

Hierbei gelten die in TABELLE 1 in Abschn. [II].(3).C dargelegten Regeln. Insbesondere ist:

| W1   | W2   | W3    | W1 & W2 & W3 |
|------|------|-------|--------------|
| NA   | TRUE | FALSE | FALSE        |
| NA   | NA   | FALSE | FALSE        |
| NA   | TRUE | TRUE  | NA           |
| NA   | NA   | NA    | NA           |
| TRUE | TRUE | TRUE  | TRUE         |

Die letzte Spalte entspricht nun also den Einträgen im “blnMask-Vektor”.

**!!! Und nur solche Kauffälle mit einem TRUE-Eintrag werden von AIce bzw. der AKS als Treffer/positiv selektiert ausgegeben !!!**

Man sieht Folgendes:

- Ein “NA”-Eintrag bedeutet, dass **mindestens ein** Kriteriums-Element nicht belegt gewesen ist, und **kein** Kriterium **aktiv verfehlt** wurde (das zugehörige Element ist belegt und verfehlt das Kriterium).
- Ein “FALSE”-Eintrag bedeutet, dass **mindesten ein** AKS-Element ein Kriterium **aktiv nicht erfüllt** (ist belegt und erfüllt ein Kriterium nicht).
- Ein “TRUE”-Eintrag bedeutet, dass alle Kriterien **aktiv erfüllt** werden (das zugehörige Element ist belegt und das Kriterium trifft zu).

### (C) Mehrfachbelegung

AIce erlaubt es (im Gegensatz zur AKS), einzelne Elemente mit mehreren Kriterien zu belegen:

**Bspw. selektiert**

```
mein_ase1$addSepa("WEIA", "*Teilstring_1*")
mein_ase1$addSepa("WEIA", "<>(*Teilstring_2*)")
```

alle Kauffälle, bei denen in **mindestens einer** WEIA-Periode der Teilstring “Teilstring\_1” vorkommt **UND** in **keiner** WEIA-Periode der Teilstring “Teilstring\_2” enthalten ist.

(Natürlich kann man entsprechend auch in den Unterelementen “WEIA1”,...,“WEIA8” selektieren.)

**Bspw. selektiert**

```
mein_ase1$addSepa("DATU", "<>01.01.2024")
mein_ase1$addSepa("DATU", "<>01.02.2024")
# man kann hier auch "<>(...)" benutzen
```

alle Kauffälle mit Kauffalldatum ungleich “01.01.2024” und ungleich “01.02.2024”.

### (D) Zusatz ~ Kürzungsregeln für “<>(...)”

Dieser Abschnitt dient dazu klar zu machen, dass die in [III].(2).(B).[e] angegebenen “Beschneidungsregeln” keine substantiellen Einschränkungen an die möglichen Selektionen liefern:

### Zu Regel 1.:

Wir haben die Regel

$$! \# \overline{\text{AUSDRUCK}} \rightarrow !$$

festgelegt, um unsinnige NAs zu vermeiden:

- Ist nämlich das entsprechende AKS-Element **belegt**, so liefern beide Seiten “TRUE” - die Komponente AUSDRUCK spielt hier dann gar keine Rolle.
- Im **anderen** Fall liefert die rechte Seite “FALSE” - was wir auch so wollen; aber die linke Seite liefert  $\text{FALSE} \# \text{NA} \hat{=} \text{NA}$  - was nicht wirklich hilfreich ist.

### Zu den Kürzungen in Regel 3.:

Ist das entsprechende AKS-Element **nicht belegt**, so erhalten wir automatisch

- “TRUE” bei einem Mischausdruck mit “=”,
- “NA” bei einem Ausdruck ohne “=”.

Hier spielen die Kürzungsregeln also sowieso keine Rolle. Wir müssen daher nur noch den Fall plausibilisieren, in dem das entsprechende AKS-Element belegt ist:

- Die Elementarbausteine  $A_1, \dots, A_n$  eines regulären AUSDRUCKs wie

$$- A_1 \# \dots \# A_n \quad \text{oder} \quad <> (A_1 \# \dots \# A_n)$$

nennen wir **Elementarkriterien**:

- wie bspw. “01.01.2024”, “01.01.2024:31.12.2024”,  $<>01.01.2024$ , etc.

Seien  $K_1, K_2$  zwei **Elementarkriterien** auf einem **belegten** AKS-Element  $E$ , die nicht beide zugleich TRUE sein können:

- Dann gilt immer  $K_1 \& K_2 = \text{FALSE}$ ; und hiermit

$$\begin{aligned} <> (K_1 \# Z) \# <> (K_2 \# Z) &\hat{=} <> (\underbrace{(K_1 \# Z) \& (K_2 \# Z)}_{Z \# (K_1 \& K_2)}) \\ &\hat{=} <> (Z) \& \overbrace{<> (K_1 \& K_2)}^{\text{TRUE}} \\ &\hat{=} <> (Z) \end{aligned}$$

- Es gilt immer  $\langle \rangle (K_1) \# K_2 \equiv \langle \rangle (K_1)$  wegen  
*(Beachte, dass der Fall  $K_1, K_2 = \text{TRUE}$  per Voraussetzung nicht auftreten kann.)*

| K1    | K2    | $\langle \rangle (K_1)$ | $\langle \rangle (K_1) \# K_2$ |
|-------|-------|-------------------------|--------------------------------|
| TRUE  | FALSE | FALSE                   | FALSE                          |
| FALSE | TRUE  | TRUE                    | TRUE                           |
| FALSE | FALSE | TRUE                    | TRUE                           |

- Insbesondere folgt:

$$\begin{aligned}
 \langle \rangle (K_1 \# K_2) \# K_2 &\equiv (\langle \rangle (K_1) \& \langle \rangle (K_2)) \# K_2 \\
 &\equiv (\langle \rangle (K_1) \# K_2) \& \underbrace{(\langle \rangle (K_2) \# K_2)}_{\text{TRUE}} \\
 &\equiv \langle \rangle (K_1) \# K_2 \\
 &\equiv \langle \rangle (K_1)
 \end{aligned}$$